Gemini and CoPilot were both given the prompt below. Both GenAIs were accessed on a KU Linux machine using Firefox browser.

Prompt:

- “Generate an object based C++ program that will prioritize emails for a CEO using a max heap to implement a priority queue. The max heap will be list based and will be used to handle all email prioritizing. Create objects from scratch, do not use preexisting heap modules. There should be detailed and understandable comments explaining each code block, function, variable initialization, or complicated section. The program will take a txt file (obtained from the user with std::cin) containing commands as input and execute them in order based on their order in the file. These commands include: EMAIL, NEXT, READ, and COUNT. EMAIL is followed by a space and will contain a <sender category>, <subject line> and <date> in that order and separated by commas. The emails will be prioritized by their sender and date. The senders from highest to lowest priority are strings named Boss, Subordinate, Peer, ImportantPerson, OtherPerson. If there is more than one email from a sender, the newest email should take priority. <subject line> is a string which can contain spaces but not commas. <date> is in the format MM-DD-YYYY. NEXT displays the next email to be read in the format “Next Email:\n Sender:\n Subject:\n Date:\n”. READ will mark the highest priority email as read by removing it from the max heap and reordering the other emails accordingly. COUNT will display the current count of unread emails.”
-

Gemini’s Program:

```
/**
* @struct Date
* @brief Parses and represents a calendar date in MM-DD-YYYY format.
*/
struct Date {
   int month;
   int day;
   int year;

   /**
    * @brief Parses a date string into numerical components.
    * @param dateStr String in "MM-DD-YYYY" format.
    * @return Populated Date struct.
    */
   static Date parse(const std::string& dateStr) {
       Date d;
       // Extract month, day, and year substrings and convert to integers
```

```
        d.month = std::stoi(dateStr.substr(0, 2));
        d.day = std::stoi(dateStr.substr(3, 2));
        d.year = std::stoi(dateStr.substr(6, 4));
        return d;
    }

    /**
     * @brief Less-than operator to compare chronological date order.
     */
    bool operator<(const Date& other) const {
        if (year != other.year) return year < other.year;
        if (month != other.month) return month < other.month;
        return day < other.day;
    }

    /**
     * @brief Greater-than operator (newer date is greater).
     */
    bool operator>(const Date& other) const {
        return other < *this;
    }
};

/**
* @class Email
* @brief Represents an individual email message with sender, subject, and date.
*/
class Email {
private:
    std::string sender;
    std::string subject;
    std::string dateStr;
    Date parsedDate;

    /**
     * @brief Assigns a numerical priority rank to sender categories.
     * @return Integer rank where higher value means higher priority.
     *         Boss (5) > Subordinate (4) > Peer (3) > ImportantPerson (2) >
OtherPerson (1)
```

```
     */
    int getSenderRank() const {
        if (sender == "Boss") return 5;
        if (sender == "Subordinate") return 4;
        if (sender == "Peer") return 3;
        if (sender == "ImportantPerson") return 2;
        if (sender == "OtherPerson") return 1;
        return 0; // Default fallback for unknown senders
    }

public:
    /**
     * @brief Constructor for Email object.
     */
    Email(std::string s, std::string subj, std::string d)
        : sender(std::move(s)), subject(std::move(subj)), dateStr(std::move(d)) {
        parsedDate = Date::parse(dateStr);
    }

    // Accessor methods
    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return dateStr; }

    /**
     * @brief Custom priority comparison operator for max heap.
     * @details Returns true if this email has higher priority than 'other'.
     *          Priority rules:
     *          1. Higher sender category rank takes precedence.
     *          2. If sender ranks are equal, the newer date takes precedence.
     */
    bool operator>(const Email& other) const {
        int thisRank = this->getSenderRank();
        int otherRank = other.getSenderRank();

        if (thisRank != otherRank) {
            return thisRank > otherRank;
        }
        // Tie-breaker: Newer date gets higher priority
```

```
        return this->parsedDate > other.parsedDate;
    }

    /**
     * @brief Less-than operator utilized by heap algorithms.
     */
    bool operator<(const Email& other) const {
        return other > *this;
    }
};

/**
* @class EmailMaxHeap
* @brief A list-based max heap implemented completely from scratch for priority queue management.
*/
class EmailMaxHeap {
private:
    // Dynamic array (list) backing the binary heap structure
    std::vector<Email> heap;

    // Helper: Get parent index of node i
    int parent(int i) { return (i - 1) / 2; }

    // Helper: Get left child index of node i
    int leftChild(int i) { return (2 * i) + 1; }

    // Helper: Get right child index of node i
    int rightChild(int i) { return (2 * i) + 2; }

    /**
     * @brief Percolates an element upward to restore the max-heap property.
     */
    void heapifyUp(int i) {
        while (i > 0 && heap[parent(i)] < heap[i]) {
            std::swap(heap[parent(i)], heap[i]);
            i = parent(i);
        }
    }
```

```
    /**
     * @brief Percolates an element downward to restore the max-heap property.
     */
    void heapifyDown(int i) {
        int maxIndex = i;
        int l = leftChild(i);
        int r = rightChild(i);
        int size = heap.size();

        // Compare with left child
        if (l < size && heap[maxIndex] < heap[l]) {
            maxIndex = l;
        }

        // Compare with right child
        if (r < size && heap[maxIndex] < heap[r]) {
            maxIndex = r;
        }

        // If a child has higher priority, swap and continue heapifying down
        if (maxIndex != i) {
            std::swap(heap[i], heap[maxIndex]);
            heapifyDown(maxIndex);
        }
    }

public:
    /**
     * @brief Inserts a new email into the max heap and reorders.
     */
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    /**
     * @brief Returns the highest priority email without removing it.
     */
```

```
    const Email& getMax() const {
        if (heap.empty()) {
            throw std::runtime_error("Heap is empty!");
        }
        return heap[0];
    }

    /**
     * @brief Removes the highest priority email (root) and reorders remaining
elements.
     */
    void removeMax() {
        if (heap.empty()) {
            return;
        }
        // Move last element to root and remove back
        heap[0] = heap.back();
        heap.pop_back();

        // Restore max-heap property from top down
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    /**
     * @brief Returns the total count of unread emails in the heap.
     */
    size_t size() const {
        return heap.size();
    }

    /**
     * @brief Checks if the heap contains any unread emails.
     */
    bool isEmpty() const {
        return heap.empty();
    }
};
```

```cpp
/**
* @brief Helper utility to trim whitespace from command substrings.
*/
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

int main() {
    // Optimize standard I/O operations for performance
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    EmailMaxHeap emailHeap;
    std::string line;

    // Read commands line-by-line from standard input (file redirected via cin)
    while (std::getline(std::cin, line)) {
        line = trim(line);
        if (line.empty()) continue;

        // 1. Handle EMAIL command: Adds an email to the priority queue
        if (line.rfind("EMAIL", 0) == 0) {
            std::string payload = line.substr(5); // Strip "EMAIL" keyword

            // Parse comma-separated fields: <sender category>, <subject line>, <date>
            size_t firstComma = payload.find(',');
            size_t secondComma = payload.find(',', firstComma + 1);

            if (firstComma != std::string::npos && secondComma != std::string::npos) {
                std::string sender = trim(payload.substr(0, firstComma));
                std::string subject = trim(payload.substr(firstComma + 1, secondComma -
firstComma - 1));
                std::string date = trim(payload.substr(secondComma + 1));

                Email newEmail(sender, subject, date);
```

```
                emailHeap.insert(newEmail);
            }
        }
        // 2. Handle NEXT command: Displays the highest priority email
        else if (line == "NEXT") {
            if (!emailHeap.isEmpty()) {
                const Email& topEmail = emailHeap.getMax();
                std::cout << "Next Email:\n";
                std::cout << " Sender: " << topEmail.getSender() << "\n";
                std::cout << " Subject: " << topEmail.getSubject() << "\n";
                std::cout << " Date: " << topEmail.getDate() << "\n";
            } else {
                std::cout << "Next Email:\n No unread emails.\n";
            }
        }
        // 3. Handle READ command: Marks highest priority email as read (removes from
heap)
        else if (line == "READ") {
            if (!emailHeap.isEmpty()) {
                emailHeap.removeMax();
            }
        }
        // 4. Handle COUNT command: Displays the current count of unread emails
        else if (line == "COUNT") {
            std::cout << emailHeap.size() << "\n";
        }
    }

    return 0;
}
```

Copilot's Program:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
```

```cpp
using namespace std;

/*
------------------------------------------------------------
Email Class
------------------------------------------------------------
Represents a single email with:
- sender category
- subject line
- date

The heap will store Email objects.
------------------------------------------------------------
*/
class Email
{
public:
   string senderCategory;
   string subject;
   string date;

   Email()
   {
       senderCategory = "";
       subject = "";
       date = "";
   }

   Email(string sender, string subj, string emailDate)
   {
       senderCategory = sender;
       subject = subj;
       date = emailDate;
   }
};

/*
------------------------------------------------------------
```

```
MaxHeap Class
------------------------------------------------------------
Uses a vector as the underlying list structure.

The highest priority email is always stored at index 0.

Priority Rules:
1. Sender category determines primary priority.
2. If sender categories match, the newer date wins.
------------------------------------------------------------
*/
class MaxHeap
{
private:

   // Internal list used to represent the heap
   vector<Email> heap;

   /*
   ------------------------------------------------------
   senderPriority()

   Converts sender category strings into numerical values.

   Higher number = higher priority.

   Boss             -> 5
   Subordinate      -> 4
   Peer             -> 3
   ImportantPerson  -> 2
   OtherPerson      -> 1
   ------------------------------------------------------
   */
   int senderPriority(const string& sender) const
   {
       if (sender == "Boss")
           return 5;
       else if (sender == "Subordinate")
           return 4;
```

```
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1;
}

/*
-------------------------------------------------------
dateValue()

Converts a date from MM-DD-YYYY into an integer
YYYYMMDD format.

Example:
03-15-2025 becomes 20250315

Larger value means newer date.
-------------------------------------------------------
*/
int dateValue(const string& date) const
{
    string month = date.substr(0, 2);
    string day   = date.substr(3, 2);
    string year  = date.substr(6, 4);

    return stoi(year + month + day);
}

/*
-------------------------------------------------------
higherPriority()

Compares two emails.

Returns true if email A should appear higher
in the max heap than email B.
```

```
Comparison Order:
1. Sender priority
2. Newer date
-----------------------------------------------------
*/
bool higherPriority(const Email& a, const Email& b) const
{
    int senderA = senderPriority(a.senderCategory);
    int senderB = senderPriority(b.senderCategory);

    // Compare sender categories first
    if (senderA != senderB)
    {
        return senderA > senderB;
    }

    // If same sender category,
    // newer date gets priority
    return dateValue(a.date) > dateValue(b.date);
}

/*
-----------------------------------------------------
heapifyUp()

Used after insertion.

Moves a newly inserted email upward until
max heap order is restored.
-----------------------------------------------------
*/
void heapifyUp(int index)
{
    while (index > 0)
    {
        int parent = (index - 1) / 2;

        if (higherPriority(heap[index], heap[parent]))
        {
```

```
            swap(heap[index], heap[parent]);
            index = parent;
        }
        else
        {
            break;
        }
    }
}

/*
-----------------------------------------------------
heapifyDown()

Used after removing the root.

Moves an email downward until max heap
ordering is restored.
-----------------------------------------------------
*/
void heapifyDown(int index)
{
    int size = heap.size();

    while (true)
    {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int largest = index;

        if (leftChild < size &&
            higherPriority(heap[leftChild],
                           heap[largest]))
        {
            largest = leftChild;
        }

        if (rightChild < size &&
            higherPriority(heap[rightChild],
```

```
                                heap[largest]))
            {
                largest = rightChild;
            }

            if (largest != index)
            {
                swap(heap[index], heap[largest]);
                index = largest;
            }
            else
            {
                break;
            }
        }
    }

public:

    /*
    ------------------------------------------------------
    insert()

    Adds a new email and restores heap order.
    ------------------------------------------------------
    */
    void insert(const Email& email)
    {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    /*
    ------------------------------------------------------
    empty()

    Returns true if heap contains no emails.
    ------------------------------------------------------
    */
```

```
bool empty() const
{
    return heap.empty();
}

/*
-----------------------------------------------------
size()

Returns number of unread emails.
-----------------------------------------------------
*/
int size() const
{
    return heap.size();
}

/*
-----------------------------------------------------
getMax()

Returns the highest priority email.

Assumes heap is not empty.
-----------------------------------------------------
*/
Email getMax() const
{
    return heap[0];
}

/*
-----------------------------------------------------
removeMax()

Removes highest priority email (READ command).

Steps:
1. Replace root with last element.
```

```
   2. Delete last element.
   3. Heapify down.
   ------------------------------------------------------
   */
   void removeMax()
   {
       if (heap.empty())
           return;

       heap[0] = heap.back();
       heap.pop_back();

       if (!heap.empty())
       {
           heapifyDown(0);
       }
   }
};

/*
------------------------------------------------------------
trim()

Removes leading and trailing spaces from strings.

Useful when parsing command fields.
------------------------------------------------------------
*/
string trim(string str)
{
   while (!str.empty() && str.front() == ' ')
       str.erase(str.begin());

   while (!str.empty() && str.back() == ' ')
       str.pop_back();

   return str;
}
```

```
/*
------------------------------------------------------------
processEmailCommand()

Parses:

EMAIL sender, subject, date

and inserts email into heap.
------------------------------------------------------------
*/
void processEmailCommand(const string& line,
                         MaxHeap& inbox)
{
   string content = line.substr(6);

   stringstream ss(content);

   string sender;
   string subject;
   string date;

   getline(ss, sender, ',');
   getline(ss, subject, ',');
   getline(ss, date);

   sender = trim(sender);
   subject = trim(subject);
   date = trim(date);

   Email newEmail(sender, subject, date);

   inbox.insert(newEmail);
}

/*
------------------------------------------------------------
Main Function
------------------------------------------------------------
```

```
1. Ask user for input filename.
2. Open file.
3. Execute commands sequentially.
4. Display requested output.
------------------------------------------------------------
*/
int main()
{
   MaxHeap inbox;

   string filename;

   cout << "Enter command file name: ";
   cin >> filename;

   ifstream inputFile(filename);

   if (!inputFile)
   {
       cout << "Error opening file." << endl;
       return 1;
   }

   string line;

   while (getline(inputFile, line))
   {
       /*
       ---------------------------------------------------
       EMAIL COMMAND
       ---------------------------------------------------
       */
       if (line.rfind("EMAIL", 0) == 0)
       {
           processEmailCommand(line, inbox);
       }

       /*
```

```
    ------------------------------------------------
    NEXT COMMAND

    Display highest priority unread email.
    ------------------------------------------------
    */
    else if (line == "NEXT")
    {
        if (inbox.empty())
        {
            cout << "No unread emails." << endl;
        }
        else
        {
            Email nextEmail = inbox.getMax();

            cout << "Next Email:" << endl;
            cout << "Sender: "
                 << nextEmail.senderCategory
                 << endl;

            cout << "Subject: "
                 << nextEmail.subject
                 << endl;

            cout << "Date: "
                 << nextEmail.date
                 << endl;
        }
    }

    /*
    ------------------------------------------------
    READ COMMAND

    Remove highest priority email.
    ------------------------------------------------
    */
    else if (line == "READ")
```

```
        {
            if (ibriefnbox.empty())
            {
                cout << "No unread emails." << endl;
            }
            else
            {
                inbox.removeMax();
            }
        }

        /*
        ----------------------------------------------------
        COUNT COMMAND

        Display unread email count.
        ----------------------------------------------------
        */
        else if (line == "COUNT")
        {
            cout << "Unread Emails: "
                 << inbox.size()
                 << endl;
        }
    }

    inputFile.close();

    return 0;
}
```

Analysis:

1. Gemini
    a. Correctness:
        i. The program would not take input from the command line or by prompting the user; it had to be modified for testing by adding user input via std::cin. When the program was given a file with an expected order of input, the email count was accurate, but it was returned as just the number with no context or label. The email highest in priority was properly displayed at the top of the inbox, and if two emails with the same priority sender existed, the newer email was displayed first. When the program was given a file with an unexpected order of input, it still worked as expected with similar output to above: it properly informs the user when the inbox is empty and also properly handles duplicate emails by ordering them next to each other in the inbox. However, when provided with an invalid file name, the program does not inform the user and just exits the program.
    b. Execution Time [O(log(n))]:
        i. The heapifying processes use recursion, which could slow the program as the EmailMaxHeap vector grows in size. The program also uses many external/headerfile functions, which may also slow program
    c. Space Complexity [O(n)]:
        i. Functions take variables by reference,which saves space by not making copies. The size of EmailMaxHeap vector is determined by user file commands, which is why the space complexity is O(n).
    d. Maintainability:
        i. The program uses classes and objects like instructed. The comments are brief and mostly one line, but they are abundant. If the email sender is not one of the 5 recognized types (unknown sender) it is set to a priority value of 0. In the EmailMaxHeap method size(), the return data type is set as size_t. This is beneficial because it accounts for inboxes with a large amount of emails.
        ii. The program does not use namespace std, which could affect readability/integration with other programs. It is also not evident why std::cin is tied to null; these "optimizations" (as stated by the program) made the program unable to take input.
2. CoPilot
    a. Correctness:
        i. The program takes user input using std::cin and then opens the file using the name provided by the user. When the program was given a file with an expected order of input, the email count was accurate. The email highest in priority was properly displayed at the top of the inbox, and if two emails with the same priority sender existed, the newer email was displayed first. When the program was given a file with an unexpected order of input, it properly handles out of order commands and informs the

user when the inbox is empty and the email count is 0. The program also properly handles duplicate emails by ordering them next to each other in the inbox. When provided with an invalid file name, the program informs the user and exits the program.

ii. Before testing the program, an error involving the “inbox” object prevented the program from running. When checking if an inbox object was empty, the program used a nonexistent class name “ibriefnbox”. This was fixed to check the program output.

b. Execution Time [O(n)]:
    i. Heapifying processes are fully iterative with while loops, less execution time than recursion
c. Space Complexity [O(n)]:
    i. Functions take variables by reference,which saves space by not making copies. The size of EmailMaxHeap vector is determined by user file commands, which makes the space complexity O(n).
d. Maintainability:
    i. The program uses classes and objects as instructed. There are less one line comments, but comments explaining large blocks of code/major functions are detailed with examples, which makes it easy to understand. The program also uses namespace std, which improves readability.

Program Selected: Copilot

1. Justification
    a. Comments are detailed and understandable, and the program properly handles both expected and unexpected user input and informs the user of file input errors. The object oriented approach also makes for better formatted code.
2. Changes Made
    a. Added comments explaining unfamiliar functions/syntax
    b. Fixed unknown class name: replaced “ibreifinbox” with “inbox” object name
    c. Added formatting for more readable output